MLPC 2026 Task 5: Challenge Report

Waji Ul Hassan Syed Rayan Ali Javed
Team Stochastic Pipelines

June 23, 2026

1 Baseline Reproduction

(a) We reproduced the provided baseline: one `DecisionTreeClassifier` (`max_depth=20`, `max_features='sqrt'`) per class wrapped in a `MultiOutputClassifier`, trained on 50,000 subsampled segments using the raw 960 features (no scaling, no PCA). For inference on the non-hidden test set (500 files), we predicted on overlapping 1-second segments (hop 0.5 s), kept only whole-second predictions, and merged consecutive active seconds into onset/offset intervals. The official `evaluate.py` script yields a Segment-based Macro F1 of **0.3170**. Per-class results are reported in Table 1.

(b) Performance varies widely, from F1 = 0.061 (window_open_close) to 0.578 (running_water). Long, continuous sounds score highest because they occupy many stable 1-second segments, whereas short transients suffer from the coarse 1-second resolution and the discarded half-second predictions. Class frequency helps but is not decisive: footsteps (≈10% of segments) reaches only 0.333, while the less frequent phone_ringing reaches 0.485, indicating that acoustic ambiguity and label noise often dominate. Mechanical impacts (door, wardrobe, window) share broadband transient spectra and are mutually confused (F1 0.061–0.177). Footsteps additionally overlaps with background transients and suffers from low annotator agreement (84.82% in Task 3). Precision generally exceeds recall, showing a conservative model that avoids false alarms but misses true events, consistent with shallow per-class trees.

(c) The baseline is simple, interpretable, and fast to train and run; it handles polyphonic audio via binary relevance and reuses precomputed acoustic features. Its main limitations are: independent per-class classifiers cannot model class co-occurrence; the 1-second resolution and discarded half-second predictions miss short transients; single unregularised trees capture only axis-aligned boundaries and struggle to separate acoustically similar classes; feature aggregation over one second discards fine-scale dynamics; and there is no temporal modelling, so predictions are noisy across time. Majority voting also propagates label noise for low-agreement classes. The baseline works well for long, distinctive sounds (running_water, vacuum_cleaner) and poorly on short, ambiguous, or noisy-label events (window_open_close, wardrobe_drawer), motivating a stronger classifier.

Table 1: Per-class baseline precision, recall, and F1 on the non-hidden test set, and the F1 of our Random Forest with the absolute change Δ. Macro F1: baseline 0.3170, Random Forest 0.4714.

Class	Base P	Base R	Base F1	RF F1	Δ
bell_ringing	0.426	0.218	0.288	0.207	−0.081
coffee_machine	0.504	0.283	0.362	0.449	+0.087
cutlery_dishes	0.294	0.281	0.287	0.489	+0.202
door_open_close	0.200	0.159	0.177	0.370	+0.193
footsteps	0.384	0.293	0.333	0.488	+0.155
keyboard_typing	0.372	0.347	0.359	0.570	+0.211
keychain	0.365	0.287	0.321	0.496	+0.175
light_switch	0.240	0.228	0.234	0.484	+0.250
microwave	0.408	0.373	0.390	0.609	+0.219
phone_ringing	0.534	0.444	0.485	0.559	+0.074
running_water	0.655	0.517	0.578	0.695	+0.118
toilet_flushing	0.288	0.292	0.290	0.521	+0.231
vacuum_cleaner	0.485	0.455	0.470	0.637	+0.167
wardrobe_drawer_open_close	0.134	0.109	0.120	0.292	+0.172
window_open_close	0.057	0.066	0.061	0.206	+0.145

2 Simple Classifiers

(a) Our best classical classifier is a Random Forest, implemented via `MultiOutputClassifier` (binary relevance, one model per class) for polyphonic SED. We chose it over single decision trees because: (i) bagging many trees on bootstrapped samples reduces variance and overfitting; (ii) random feature selection at each split decorrelates the trees and improves generalisation; (iii) `class_weight='balanced'` upweights minority classes, which is critical given that the rarest classes occupy under 2% of segments; and (iv) `max_depth` and `min_samples_leaf` provide explicit regularisation. The final model uses `n_estimators=200`, `max_depth=20`, `min_samples_leaf=5`, and `class_weight='balanced'`, trained on the same 50,000 segments with raw features (no scaling, no PCA, matching the baseline for a fair comparison). Trees are scale-invariant, so scaling and PCA are unnecessary and can even harm axis-aligned splits.

(b) We replaced the decision trees with the Random Forest and studied the effect of `max_depth` and `min_samples_leaf` on validation Macro F1, with `n_estimators=200` and balanced weighting fixed (Figure 1). Tuning was performed on the local validation set only, never the non-hidden test set. Raising `min_samples_leaf` from 1 to 5 helped at every depth (e.g. 0.4633 to 0.4818 at depth 20). Performance rose from depth 10 to 20 (0.4720 to 0.4818) as deeper trees captured more complex patterns, then fell at depth 30 (0.4512) from overfitting. We selected `max_depth=20`, `min_samples_leaf=5` (validation Macro F1 0.4818).

(c) On the non-hidden test set, the Random Forest raises Macro F1 from 0.3170 to **0.4714** (+0.1544), evaluated with the same official metric and pipeline

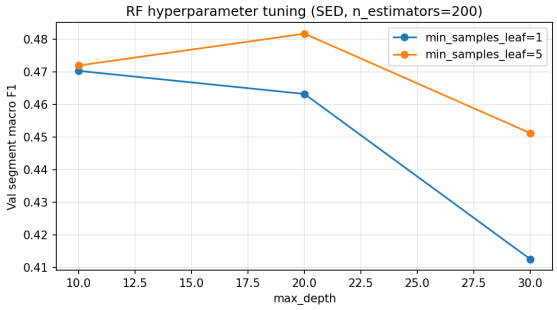

Figure 1: Random Forest tuning: validation Macro F1 versus `max_depth` for `min_samples_leaf` $\in \{1, 5\}$ (`n_estimators=200`, balanced).

as the baseline (Table 1). The largest gains are on previously weak classes: light_switch (+0.250), toilet_flushing (+0.231), microwave (+0.219), keyboard_typing (+0.211), and cutlery_dishes (+0.202). Balanced weighting plus ensemble stability recovers rare and mid-frequency classes that a single tree failed on. The hard mechanical-transient classes improve in relative terms but remain lowest: window_open_close (0.206) and wardrobe_drawer_open_close (0.292), reflecting the 1-second resolution limit. One class regresses: bell_ringing drops −0.081 (0.288 to 0.207), likely because balanced weighting makes the model over-predict competing rare classes in those segments, compounded by its scarcity and tonal similarity to phone_ringing.

(d) We analysed two test files (Figures 2 and 3). In `000790.wav`, running_water is detected almost perfectly (14 of 15 true seconds, no false alarms), confirming that continuous, energetic sounds are handled well. However, microwave (TP = 4, FN = 10) and phone_ringing (TP = 3, FN = 6) show low recall: the model fires on their salient frames but misses the quieter sustained or inter-ring portions. Three false cutlery_dishes seconds appear, a mild instance of balanced-weighting over-prediction. In `005294.wav`, running_water (no false alarms) and toilet_flushing are again caught, but the mechanical transients are missed or smeared: wardrobe_drawer (5 of 8 true seconds missed) and door_open_close (FP = 3), with extra false positives on footsteps (FP = 4) and other classes. These two files capture all our error modes: reliable detection of continuous sounds, low recall on intermittent sounds, confusion among short mechanical transients, and over-prediction from balanced weighting.

3 Post-Processing and Temporal Refinement

(a) Because each 1-second segment is classified independently, a class's predicted activity can be temporally noisy. We applied *median filtering* to each class's per-second prediction sequence, replacing every value with the median of a centred temporal window. The filter was applied per recording, per class, to the whole-

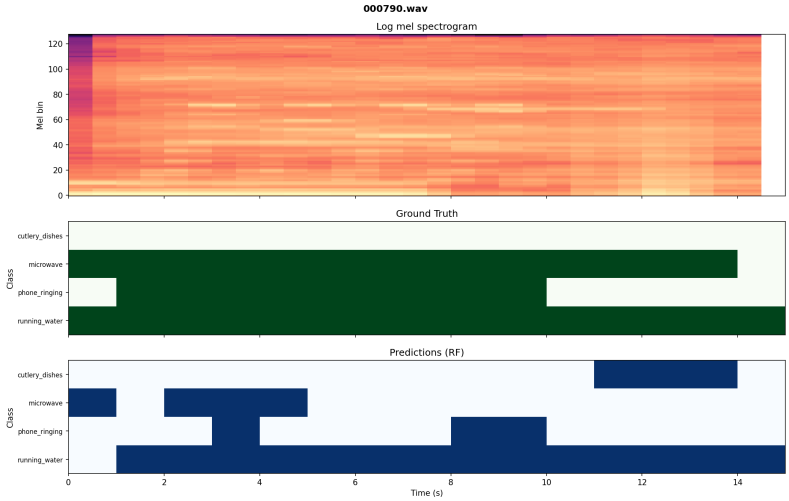

Figure 2: `000790.wav`: log-mel spectrogram (top), ground truth (middle), and Random Forest predictions (bottom). running_water is detected reliably; microwave and phone_ringing show low recall.

second prediction matrix *before* merging into intervals, so smoothing operates on each file's own sequence. We swept the key parameter, the filter window size, over $\{1, 3, 5, 7, 9\}$ seconds (window 1 = no filtering), evaluating each on the non-hidden test set with the official metric.

(b) Median filtering did not improve performance: Macro F1 decreased monotonically from 0.4714 (no filtering) to 0.3638 at window 9 (Table 2, Figure 4). The best configuration is therefore no post-processing. The cause lies in our error profile, not the technique. Median filtering helps only when errors are short, isolated false activations within otherwise correct predictions. As shown in Section 2(d), our dominant error is instead low recall on intermittent events (microwave 4 of 14, phone_ringing 3 of 9). When true detections are already sparse and broken up (e.g. `0 1 0 0 1 0`), the local median erases the genuine detections rather than removing noise, lowering recall further. Additionally, many target events (door, wardrobe, window) are 1–2 second transients, and any window of size ≥ 3 removes events shorter than half the window; since Macro F1 weights classes equally, destroying these short rare-class events disproportionately lowers the average. Larger windows amplify both effects, explaining the monotonic decline. We retain the unfiltered Random Forest as our final system. This is a meaningful negative result: temporal smoothing is beneficial only when the residual errors match the assumptions of the smoothing operator, which they do not here.

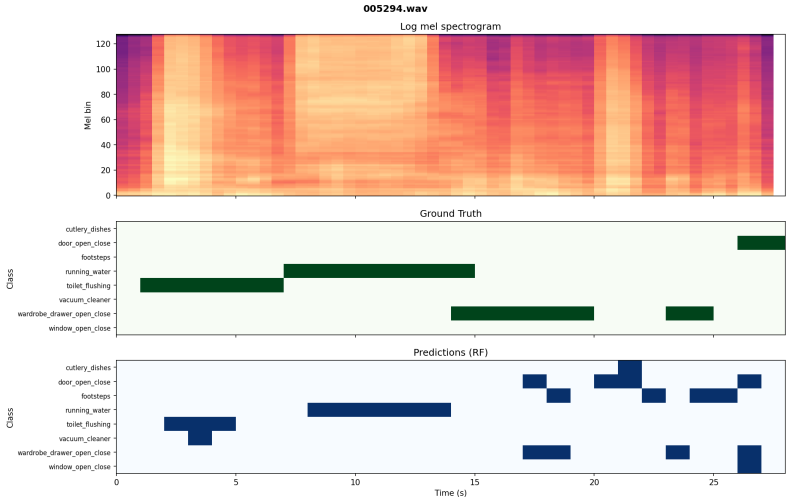

Figure 3: `005294.wav`: continuous sounds (running_water, toilet_flushing) are caught, while short mechanical transients are missed or confused and several false positives appear.

Table 2: Effect of median filter window size on non-hidden test Macro F1.

Window (s)	1 (none)	3	5	7	9
Macro F1	**0.4714**	0.4432	0.4071	0.3842	0.3638

4 Reflection and Real-World Considerations

(a) The key technical limitations, grounded in our observed errors, are as follows. *Acoustic confusion between short mechanical transients*: the lowest-scoring classes (window 0.21, wardrobe 0.29, door 0.37) share broadband transient spectra and are mutually confused; a model that sees several adjacent segments at once (e.g. a CRNN) could use context to disambiguate them. *Low recall on intermittent events*: microwave and phone_ringing are detected only on their salient seconds because per-second classification has no notion of event continuity; sequence models that carry state across time (recurrent layers, or a learned smoothing stage rather than a fixed median filter) would bridge these gaps without erasing detections. *Rare classes and noisy labels*: light_switch is limited by scarce data and footsteps by label noise (84.82% agreement), pointing to more data and noise-robust training. *Over-prediction from balanced weighting*: this raised rare-class recall but introduced false positives, so a per-class decision threshold tuned on validation, rather than a single 0.5, could restore precision. *No class co-occurrence modelling*: binary relevance cannot exploit correlations such as cutlery_dishes with running_water in a kitchen.

(b) The system is not yet reliable enough for deployment as a smart-home

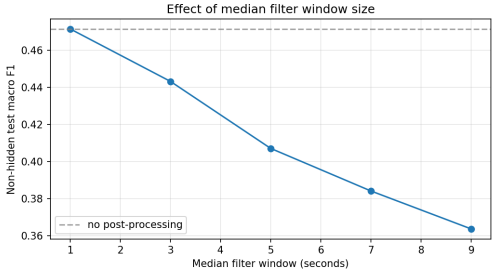

Figure 4: Non-hidden test Macro F1 versus median filter window size. The dashed line marks the no-filtering reference.

assistant, though parts are promising. On the positive side it is computationally light: a Random Forest over precomputed 1-second features runs quickly on modest hardware without a GPU, and the 1-second design supports near-real-time, low-latency operation, all desirable for an embedded device. However, an overall Macro F1 of 0.4714 means many events are missed or falsely triggered, and in a smart home a missed "running water" or a false "door opened" alert quickly erodes user trust. Robustness is unproven: real homes add reverberation, overlapping speech, music, and device-specific microphones not fully represented in training, and we already saw running_water missed in one file whose acoustics differed from training. The over-prediction behaviour would surface as frequent false alarms, the fastest way for users to disable a feature. Finally, deployment requires the exact feature-extraction front-end on-device, and any capture mismatch degrades performance. In summary, the system is an efficient proof of concept, but it would need stronger temporal models, per-class threshold calibration, and validation on realistic in-home audio before it could be trusted in a product.

Disclosure of LLM and AI Tool Use

We used Claude (Anthropic) for code generation (the SED inference pipeline, evaluation wrappers, hyperparameter sweeps, and matplotlib visualisations), for brainstorming the post-processing analysis, and for improving the clarity of this report. We also used it to cross-check both team members' notebooks for methodological consistency (ensuring all results use the official `evaluate.py` metric and that hyperparameters were tuned only on the validation set). All code was verified cell by cell against expected outputs, and every reported number was reproduced on our shared pipeline before inclusion. No content was included without being understood and verified by the authors.